



Optimism OPCMv2

Security Review

Cantina Managed review by:

Giovanni Di Siena, Lead Security Researcher

MiloTruck, Lead Security Researcher

June 8, 2026

Contents

1	Introduction	2
1.1	About Cantina	2
1.2	Disclaimer	2
1.3	Risk assessment	2
1.3.1	Severity Classification	2
2	Security Review Summary	3
2.1	Scope	3
3	Findings	4
3.1	Medium Risk	4
3.1.1	OPContractsManagerMigrator::migrate fails to re-initialize SystemConfig with migrated DelayedWETH proxy address	4
3.2	Low Risk	4
3.2.1	Dirty upper bits in SystemConfig::batcherHash will cause upgrades to revert	4
3.2.2	OPContractsManagerMigrator::migrate does not enforce SuperchainConfig version floor	4
3.2.3	Initial deployment does not validate startingRespectedGameType against enabled game set	5
3.2.4	Invalid game configs can be supplied during migration and enabled state is not respected	5
3.2.5	Repeated chain migration clears shared DisputeGameFactory implementations and breaks unmigrated portals	6
3.2.6	Calls to deploy() can be forced to revert via front-running	6
3.3	Informational	7
3.3.1	Missing NatSpec documentation in VerifyOPCM	7
3.3.2	Guardian pause can block OPCmv2 upgrades when the OPTIMISM_PORTAL_INTEROP dev feature is enabled	7
3.3.3	VerifyOPCM::runSingle does not execute the setUp() function	7
3.3.4	VerifyOPCM::_findChar is not used and can be removed	7
3.3.5	A subset of game types are not cleared during migration	8
3.3.6	Partial migration can result in loss of access to shared ETHLockbox liquidity for non-migrated portals	8
3.3.7	Duplicate upgrade instructions can cause silent key shadowing	8
3.3.8	OPContractsManagerMigrator should pass the correct AddressManager to proxy deployment args	9
3.3.9	Inaccurate PERMIT_ALL_CONTRACTS_INSTRUCTION comment	9
3.3.10	Shared migration anchors governance configuration to chain 0	9
3.3.11	loadBytes() returns empty bytes instead of reverting for addresses with no code	10
3.3.12	AnchorStateRegistry.isGameRegistered() no longer explicitly checks a dispute game's ASR address	10
3.3.13	Potential footgun when re-initialize ETHLockbox with only its own OptimismPortal	10
3.3.14	OPContractsManagerV2.migrate() should only be callable in testing environments	11
3.3.15	Potential risk with StorageSetter upgrade pattern	11

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

Optimism is a fast, stable, and scalable L2 blockchain built by Ethereum developers, for Ethereum developers. Built as a minimal extension to existing Ethereum software, Optimism's EVM-equivalent architecture scales your Ethereum apps without surprises. If it works on Ethereum, it works on Optimism at a fraction of the cost.

From Feb 8th to Feb 18th the Cantina team conducted a review of [optimism](#) on commit hash [a00b3972](#). The team identified a total of **22** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	1	1	0
Low Risk	6	5	1
Gas Optimizations	0	0	0
Informational	15	15	0
Total	22	21	1

2.1 Scope

The security review had the following components in scope for [optimism](#) on commit hash [a00b3972](#):

```
packages/contracts-bedrock
├─ scripts/deploy/VerifyOPCM.s.sol
├─ src/L1/opcm
│   ├── OPContractsManagerContainer.sol
│   ├── OPContractsManagerMigrator.sol
│   ├── OPContractsManagerUtils.sol
│   ├── OPContractsManagerUtilsCaller.sol
│   └─ OPContractsManagerV2.sol
```

3 Findings

3.1 Medium Risk

3.1.1 `OPContractsManagerMigrator::migrate` fails to re-initialize `SystemConfig` with migrated `DelayedWETH` proxy address

Severity: Medium Risk

Context: `OPContractsManagerMigrator.sol#L190-L196`, `OPContractsManagerV2.sol#L496-L504`, `OPContractsManagerV2.sol#L847-L858`

Description: `OPContractsManagerMigrator::migrate` first deploys new proxies for the `ETHLockbox`, `DisputeGameFactory`, `AnchorStateRegistry`, and `DelayedWETH` contracts before initializing through `_upgrade()` calls. Unlike the other proxies that are obtained within `OPCMv2 _loadChainContracts()` via the `OptimismPortal2` source address, `DelayedWETH` and `DisputeGameFactory` are obtained from the `SystemConfig`.

While slightly more convoluted, the resolution of `DisputeGameFactory` is correct given that it is obtained directly from the `AnchorStateRegistry` reference updated within the call to `OptimismPortalInterop::migrateToSuperRoots`; however, `SystemConfig` is never re-initialized to reference the new `DelayedWETH` proxy address. As such, `OPCMv2` will continue to reference the legacy pre-migration proxy address which will cause issues for future upgrades. Consider the following scenario:

1. `_loadChainContracts()` pulls the legacy `DelayedWETH` contract from `SystemConfig`.
2. `_apply()` rewrites `DisputeGameFactory` game args using the legacy `DelayedWETH` address.
3. Since migration creates a shared `DisputeGameFactory` for the interop set, this rewrite affects all chains using the shared factory, not just the chain that was upgraded.
4. All subsequent games will use a different (legacy) `DelayedWETH` contract compared with games created immediately after migration.

Recommendation: Re-initialize `SystemConfig` by updating the `DELAYED_WETH_SLOT` storage slot.

OP Labs: Fixed in [PR 19281](#).

Cantina Managed: Verified. The existing `DelayedWETH` from `chainSystemConfigs[0]` is now reused rather than deploying a new one.

3.2 Low Risk

3.2.1 Dirty upper bits in `SystemConfig::batcherHash` will cause upgrades to revert

Severity: Low Risk

Context: `OPContractsManagerV2.sol#L560-L568`, `SystemConfig.sol#L115-L118`, `SystemConfig.sol#L365-L378`

Description: `OPContractsManagerV2::_loadFullConfig` decodes the batcher as an address, but `SystemConfig::batcherHash` is a `bytes32` public variable and so its getter returns raw ABI-encoded `bytes32`. The dev comment states it is represented as an address left-padded with zeros to 32 bytes; however, `SystemConfig::setBatcherHash` accepts arbitrary `bytes32` from the owner without enforcement. This can result in `batcherHash` having non-zero upper bytes which will revert the `OPCMv2 upgrade()` call for the chain during this decode step until overridden by the owner.

Recommendation: Consider explicitly zeroing out the potentially dirty upper 96 bits.

OP Labs: We've made the decision that `OPCMv2` forcing this to be an address is actually safer behavior because non-address batcher hash is not actually supported in the specification for op-node. Although the variable is `bytes32`, it's far safer for `OPCMv2` to restrict to the officially supported configuration.

Cantina Managed: Acknowledged.

3.2.2 `OPContractsManagerMigrator::migrate` does not enforce `SuperchainConfig` version floor

Severity: Low Risk

Context: `OPContractsManagerMigrator.sol#L85-L88`, `OPContractsManagerV2.sol#L708-L711`

Description: `OPContractsManagerMigrator` currently only validates that all chains reference the same `SuperchainConfig` address but, unlike `OPContractsManagerV2::_apply`, does not enforce that it is upgraded to the latest version. This can allow migration to proceed with an outdated `SuperchainConfig` and potentially unsupported prestate.

Recommendation: Consider adding a similar version floor check in `OPContractsManagerMigrator::migrate` and revert when the shared `SuperchainConfig` is behind the target implementation version.

OP Labs: Fixed in [PR 19281](#).

Cantina Managed: Verified. It is now explicitly documented that `migrate()` does not enforce a `SuperchainConfig` version floor.

3.2.3 Initial deployment does not validate `startingRespectedGameType` against enabled game set

Severity: Low Risk

Context: `OPContractsManagerV2.sol#L671-L678`, `OPContractsManagerV2.sol#L681-L686`, `OPContractsManagerV2.sol#L838-L853`

Description: `OPContractsManagerV2` restricts the dispute game type of initial deployments to `PERMISSIONED_CANNON` as enforced within `_assertValidFullConfig()`; however, it is nowhere validated that `_cfg.startingRespectedGameType` is consistent with this restriction. A caller could pass `startingRespectedGameType = CANNON` (or any other game type) while only `PERMISSIONED_CANNON` has an implementation registered in the `DisputeGameFactory`, initializing the `AnchorStateRegistry` with a respected game type that has no corresponding implementation.

Recommendation: Consider having `_assertValidFullConfig()` also assert that `startingRespectedGameType == PERMISSIONED_CANNON` during initial deployments.

OP Labs: Fixed in [PR 19272](#).

Cantina Managed: Verified. `startingRespectedGameType` is now validated against enabled game configs such that deployment reverts with `OPContractsManagerV2_InvalidGameConfigs()` for disabled game types.

3.2.4 Invalid game configs can be supplied during migration and enabled state is not respected

Severity: Low Risk

Context: `FaultDisputeGame.sol#L850-L856`, `OPContractsManagerMigrator.sol#L204-L212`, `OPContractsManagerUtils.sol#L374-L385`, `OptimismPortalInterop.sol#L480-L485`

Description: Unlike `OPContractsManagerV2::_assertValidFullConfig` which validates game types, ordering, and bond constraints, `OPContractsManagerMigrator::migrate` accepts arbitrary `disputeGameConfigs` without validation. The only constraint is that the staticcall to `OPContractsManagerUtils::getGameImpl` must recognize the game type, otherwise it will bubble up a revert for unsupported types.

One impact is that non-super game types can be registered even though the invocation of `_makeGameArgs()` hardcodes `l2ChainId` as 0, meaning that `FaultDisputeGame` will revert due to chain ID mismatch. This logic also ignores this enabled flag and unconditionally registers each supplied game type in the new `DisputeGameFactory`, meaning operators performing interoperability migration may unintentionally activate dispute-game types they believed were disabled.

Recommendation: Add explicit validation to reject incompatible configurations containing non-super game types. Additionally ensure that only the explicitly enabled game types are registered within the `DisputeGameFactory`.

OP Labs: Fixed in [PR 19281](#).

Cantina Managed: Verified. Documentation has been added to explain why migration game config validation is deliberately minimal.

3.2.5 Repeated chain migration clears shared `DisputeGameFactory` implementations and breaks unmigrated portals

Severity: Low Risk

Context: `OPContractsManagerMigrator.sol#L229`, `OPContractsManagerMigrator.sol#L240-L247`

Description: `OPContractsManagerMigrator::migrate` currently assumes it is always operating on a legacy per-chain `DisputeGameFactory` and does not prevent repeated migration. For already migrated portals, `portal.disputeGameFactory()` resolves to the shared `DisputeGameFactory` corresponding to the shared `AnchorStateRegistry`; however, subsequent logic then unconditionally clears several game types before pointing the portal to a newly deployed `AnchorStateRegistry`. This corrupts the shared factory that other portals continue to use, preventing new games from being created.

Recommendation: While it is understood that the migrator logic is not yet production-ready, consider explicitly preventing repeated chain migration.

OP Labs: Fixed in PR 19271.

Cantina Managed: Verified. Documentation has been added to communicate the lack of support for re-migration.

3.2.6 Calls to `deploy()` can be forced to revert via front-running

Severity: Low Risk

Context: `OPContractsManagerV2.sol#L347`

Description: In `OPContractsManagerV2`, the following functions are meant to be delegate-called by the `ProxyAdminOwner`:

- `upgradeSuperchain()`.
- `upgrade()`.
- `migrate()`.

On the other hand, `deploy()` is meant to be called directly via a regular external call. However, the contract does not implement any form of checks to ensure that the functions are called as intended.

More importantly, `deploy()` calls `_loadChainContracts()`, which deploys proxy contracts using `create2` based on a user-provided `_l2ChainId` and `_saltMixer`. For example:

```
if (isInitialDeployment) {
    // Deploy the ProxyAdmin.
    proxyAdmin = IProxyAdmin(
        Blueprint.deployFrom(
            blueprints().proxyAdmin,
            _computeSalt(_l2ChainId, _saltMixer, "ProxyAdmin"),
            abi.encode(address(this))
        )
    );
};
```

This introduces a risk of an attacker front-running a user to call `deploy()` with the same `saltMixer` and `l2ChainId` to deploy contracts to the same address (but different `proxyAdminOwner`). If this occurs, the victim's `deploy()` transaction will revert as `create2` cannot deploy a proxy contract to the same address.

Recommendation: Consider the following:

- In `deploy()`, hash `saltMixer` with `msg.sender` to prevent different callers from deploying contracts to the same address.
- Adding a `onlyDelegateCall` check (which existed in OPCM v1) to `upgradeSuperchain()`, `upgrade()` and `migrate()`.

OP Labs: Fixed in PR 19272.

Cantina Managed: Verified. Upgrade/migration calls are now enforced with `onlyDelegateCall()` and `msg.sender` has been added to the deployment salt to prevent cross-caller CREATE2 collisions.

3.3 Informational

3.3.1 Missing NatSpec documentation in VerifyOPCM

Severity: Informational

Context: [VerifyOPCM.s.sol#L110-L119](#)

Description: The `OpcmContractRef` struct is currently missing NatSpec `@param` documentation for the `blueprint` member.

Recommendation: Update the NatSpec documentation to include the `blueprint` boolean.

OP Labs: Fixed in commit [5161204](#).

Cantina Managed: Verified.

3.3.2 Guardian pause can block OPCMV2 upgrades when the OPTIMISM_PORTAL_INTEROP dev feature is enabled

Severity: Informational

Context: [OPContractsManagerV2.sol#L764-L770](#), [SystemConfig.sol#L583-L589](#)

Description: While it is understood that portal interop is a dev feature that is not yet production ready, when `OPTIMISM_PORTAL_INTEROP` is enabled, `SystemConfig::setFeature` with `ETH_LOCKBOX` will revert if the system is paused at the time of upgrade. This allows a guardian pause to block OPCMV2 upgrades that enable interoperability.

Recommendation: Consider whether it is intended for upgrades to revert when the system is in a paused state.

OP Labs: Fixed in [PR 19271](#).

Cantina Managed: Verified. Guardian pause blocking interop upgrades has been explicitly documented as acceptable for a dev feature.

3.3.3 VerifyOPCM::runSingle does not execute the setUp() function

Severity: Informational

Context: [VerifyOPCM.s.sol#L287-L301](#), [VerifyOPCM.s.sol#L307-L310](#)

Description: Unlike `VerifyOPCM::run`, the standalone `VerifyOPCM::runSingle` does not execute the `setUp()` function but still depends on state referenced within `_buildArtifactPath()` and `_verifyStandardValidatorArgs()`.

Recommendation: Implement the same setup logic within `runSingle()` as in `run()` to avoid having standalone execution with uninitialized mappings producing incorrect artifact resolution or false verification failures.

OP Labs: Fixed in commit [56ee47e](#).

Cantina Managed: Verified.

3.3.4 VerifyOPCM::_findChar is not used and can be removed

Severity: Informational

Context: [VerifyOPCM.s.sol#L1611](#)

Description: `VerifyOPCM::_findChar` does not currently appear to be used.

Recommendation: Consider removing the unused `_findChar()` function.

OP Labs: Removed in OPCMV1 cleanup. Fixed in [PR 19792](#).

Cantina Managed: Verified. The function is no longer present.

3.3.5 A subset of game types are not cleared during migration

Severity: Informational

Context: `OPContractsManagerMigrator.sol#L240-L247`

Description: `Types.sol` defines additional game types that are not cleared during migration: `ASTERISC` (2), `ASTERISC_KONA` (3), `OP_SUCCINCT` (6), `SUPER_ASTERISC_KONA` (7), `OPTIMISTIC_ZK_GAME_TYPE` (10), `KAILUA` (1337).

If any of these had implementations set in the old `DisputeGameFactory`, they would remain active post-migration and new games of those types could still be created through the old proxy. While it is understood that this is not currently possible, and also that these old games cannot affect the new system's anchor state, this implementation seems to contradict the dev comment.

Recommendation: Consider clearing all possible game type implementations during migration.

OP Labs: Fixed in PR 19281.

Cantina Managed: Verified. It is now explicitly documented that hardcoded game type lists in `_assertValidFullConfig()` and `_migratePortal()` are intentional and must be kept in sync when new types are added.

3.3.6 Partial migration can result in loss of access to shared `ETHLockbox` liquidity for non-migrated portals

Severity: Informational

Context: `OPContractsManagerMigrator.sol#L198-L201`

Description: `OPContractsManagerMigrator::migrate` transfers the full `ETH` balance of a portal's current `ETHLockbox` into a newly deployed shared lockbox. While it is understood that the current intended use is for `N` independent chains to be merged into a single chain, if the old lockbox is shared by multiple portals while the migration batch includes only a subset of these chains, the migrator still transfers all pooled liquidity. If, for example, five portals initially share the same lockbox, but only three of these chains are migrated, the remaining two unmigrated chains will not be able to access their share of the pooled liquidity as it has already been transferred to the new shared lockbox.

Recommendation: Consider documenting these assumptions and edge cases more clearly.

OP Labs: Fixed in PR 19271.

Cantina Managed: Verified. Documentation has been added to communicate the lack of support for partial migration.

3.3.7 Duplicate upgrade instructions can cause silent key shadowing

Severity: Informational

Context: `OPContractsManagerV2.sol#L288-L292`

Description: `OPContractsManagerV2::_assertValidUpgradeInstructions` validates each instruction individually but does not reject duplicates. The only currently permitted instruction is `{PermittedProxyDeployment, DelayedWETH}` which is a completely different key namespace from the config override keys (`overrides.cfg.*`) used by `_loadBytes()`. Duplicates of this instruction are harmless since `_loadOrDeployProxy()` checks existence and `getInstructionByKey()` returns the first match. However, if a future version permits an instruction whose key overlaps with a `_loadBytes()` override key, duplicate entries with the same key but different data would silently shadow each other, with only the first taking effect.

Recommendation: Consider rejecting duplicate keys as a defensive measure before new instruction types are added.

OP Labs: Fixed in PR 19272.

Cantina Managed: Verified. Upgrades now revert with `OPContractsManagerV2_DuplicateUpgradeInstruction()` when duplicate non-`PermittedProxyDeployment` instruction keys are provided.

3.3.8 OPContractsManagerMigrator should pass the correct AddressManager to proxy deployment args

Severity: Informational

Context: OPContractsManagerMigrator.sol#L96, OPContractsManagerMigrator.sol#L101-L107, OPContractsManagerUtils.sol#L259-L266

Description: The assumption within OPContractsManagerMigrator::migrate that the actual addressManager address is not required as part of the proxyDeployArgs is true for the current implementation, although OPContractsManagerUtils::loadOrDeployProxy does require it for the L1CrossDomainMessenger legacy ResolvedDelegateProxy special case. While it is not currently relevant, given that this proxy is not migrated, it is worth calling out as an example of how the use of fixed named instructions (ETHLockbox, DisputeGameFactory, AnchorStateRegistry, DelayedWETH) could be preferred over PERMIT_ALL_CONTRACTS_INSTRUCTION. The current usage is fail-open, so if another _loadOrDeployProxy() call that overlooks this requirement is later added to the migrator, deployment of a potentially misconfigured proxy would be automatically permitted.

Recommendation: Consider passing the real AddressManager address to the proxy deployment args.

OP Labs: Fixed in commit 56ee47e.

Cantina Managed: Verified. The real AddressManager is now passed.

3.3.9 Inaccurate PERMIT_ALL_CONTRACTS_INSTRUCTION comment

Severity: Informational

Context: OPContractsManagerMigrator.sol#L101-L107, Constants.sol#L53-L55

Description: The inline documentation relating to the PERMIT_ALL_CONTRACTS_INSTRUCTION constant currently states that this special value is only used for deployments, however this is inaccurate since it is also used during migration.

Recommendation: Consider updating the comment or, perhaps more preferably, restricting usage within migration.

OP Labs: Fixed in PR 19271.

Cantina Managed: Verified. The comment has been updated to indicate use for both initial deployments and migrations.

3.3.10 Shared migration anchors governance configuration to chain 0

Severity: Informational

Context: OPContractsManagerMigrator.sol#L163

Description: When migrating N chains, the new shared ETHLockbox, AnchorStateRegistry, and DelayedWETH are all initialized with the first chain's SystemConfig. Validation checks that all chains share the same proxyAdmin().owner() but not the same proxyAdmin(). It is acknowledged by an inline comment that different chains may have different ProxyAdmin contracts, although it is assumed to be fine so long as the ownership validation holds.

That said, if the first chain's SystemConfig references a different ProxyAdmin than another chain's, the access control on the shared contracts would be anchored to only one chain's ProxyAdmin. Different ProxyAdmins with the same owner would pass validation but could diverge post-migration, for example if ownership of one is transferred. Furthermore, the SystemConfig address of migrated portals remains the same, meaning that it could point to a different address from the SystemConfig address used by the shared ETHLockbox and AnchorStateRegistry.

Recommendation: Consider refactoring, or at least clearly documenting, this fragile coupling between ProxyAdmin owners and SystemConfig addresses for migrated chains. For example, if different ProxyAdmin owners and SystemConfig addresses must be supported, then it may be preferable to define a canonical governance owner and SystemConfig address for shared contracts rather than implicitly relying on the zeroth element.

OP Labs: Fixed in PR 19271.

Cantina Managed: Verified. The intentional use of `chainSystemConfigs[0]` for shared contracts has been explicitly documented.

3.3.11 `loadBytes()` returns empty bytes instead of reverting for addresses with no code

Severity: Informational

Context: `OPContractsManagerUtils.sol#L198`

Description: In `OPContractsManagerUtils`, `loadBytes()` performs a low-level staticcall to the `_source` address and returns `result` if the call is successful:

```
// Otherwise, load the data from the source contract.
(bool success, bytes memory result) =
    → address(_source).staticcall(abi.encodePacked(_selector));
if (!success) {
    revert OPContractsManagerUtils_ConfigLoadFailed(_name);
}
```

However, if `_source` is an address with no code, `loadBytes()` doesn't revert but instead returns an empty `result` bytes. This could cause issues when using `loadBytes()` to fetch values from a `_source` which has not been deployed.

Note that this is not an issue in the current implementation since the result of `loadBytes()` is always passed to `abi.decode()`, which would always revert for empty bytes.

Recommendation: Consider checking that the `_source` address has code as well.

OP Labs: Fixed in [PR 19272](#).

Cantina Managed: Verified. `loadBytes()` now reverts with `OPContractsManagerUtils_ConfigLoadFailed()` if the address has no code.

3.3.12 `AnchorStateRegistry.isGameRegistered()` no longer explicitly checks a dispute game's ASR address

Severity: Informational

Context: `AnchorStateRegistry.sol#L195-L205`

Description: In the previous version of `AnchorStateRegistry` (i.e. `op-contracts/v6.0.0-rc.2`), `isGameRegistered()` checked that `_game` uses `address(this)` as its `AnchorStateRegistry`, which invalidates all games when a new `AnchorStateRegistry` is deployed:

```
// Return whether the game is factory registered and uses this AnchorStateRegistry. We
// check for both of these conditions because the game could be using a different
// AnchorStateRegistry if the registry was updated at some point. We mitigate the risks
→ of
// an outdated AnchorStateRegistry by invalidating all previous games in the initializer
→ of
// this contract, but an explicit check avoids potential footguns in the future.
return address(factoryRegisteredGame) == address(_game) && asr == address(this);
```

This check no longer exists in the current `AnchorStateRegistry`. While removing this check is safe since `retirementTimestamp` is set to `block.timestamp` in `initialize()`, which would invalidate all old dispute games, consider keeping this check to explicitly invalidate old games.

Recommendation: Consider if the check should be re-added.

OP Labs: Fixed in [PR 19281](#).

Cantina Managed: Verified. Explicit documentation of the theoretical risk has been added.

3.3.13 Potential footgun when re-initialize `ETHLockbox` with only its own `OptimismPortal`

Severity: Informational

Context: `OPContractsManagerV2.sol#L754-L761`

Description: In `OPContractsManagerV2._apply()`, a chain's `ethLockbox` is initialized with only its own `OptimismPortal` as the `portals` argument:

```
IOptimismPortal[] memory portals = new IOptimismPortal[](1);
portals[0] = _cts.optimismPortal;
_upgrade(
    _cts.proxyAdmin,
    address(_cts.ethLockbox),
    impls.ethLockboxImpl,
    abi.encodeCall(IETHLockbox.initialize, (_cts.systemConfig, portals))
);
```

However, this is a potential footgun which could very easily introduce a bug when upgrading the `ETHLockbox` implementation in the future. For example:

- If an `authorizedPortals[_portal] == false` sanity check was added in `ETHLockbox._authorizePortal()`, the call here would revert.
- What if the existing lockbox has multiple portals authorized? If `ETHLockbox.initialize()` expects all authorized lockboxes to be passed into it, this argument would be wrong.

In general, the `OPCM` introduces the assumption that re-initializing `ETHLockbox` with only its own portal does not have any side-effects, which may not always be true in the future.

Note that this is not an issue in the current implementation.

Recommendation: An implementation which *may* be safer would be to fetch all existing portals from `ETHLockbox` and pass them into `initialize()`.

Alternatively, consider adding a warning in `ETHLockbox` about this potential footgun.

OP Labs: The triage decision was to add review rules (not contract code changes) to catch this class of issue - `initialize()` functions that are not safe to re-run because they increment counters, append to arrays, make non-idempotent external calls, or overwrite variables that other systems depend on. Fixed in [PR 19273](#).

Cantina Managed: Verified. Sufficient documentation has been added.

3.3.14 `OPContractsManagerV2.migrate()` should only be callable in testing environments

Severity: Informational

Context: [OPContractsManagerV2.sol#L266](#)

Description/Recommendation: In the current version of the `OPCM`, migration functionality (i.e., `OPContractsManagerV2.migrate()`) is in development and isn't production-ready yet; it currently exists for testing out interop basics in integration tests and devnets.

As such, `migrate()` should explicitly check that the contract is running in a testing environment to prevent it from being called onchain. This can be achieved by making `OPContractsManagerContainer._isTestingEnvironment()` public and checking it.

OP Labs: Fixed in [PR 19285](#).

Cantina Managed: Verified. `OPContractsManagerMigrator.migrate()` now reverts with `OPContractsManagerMigrator_InteropNotEnabled()` if the `OPTIMISM_PORTAL_INTEROP` dev feature is not enabled in the contracts container.

3.3.15 Potential risk with `StorageSetter` upgrade pattern

Severity: Informational

Context: [OPContractsManagerUtils.sol#L330](#)

Description: In `OPContractsManagerUtils.upgrade()`, contracts are upgraded with the following process:

1. Upgrade the proxy's implementation to `StorageSetter`.
2. Zero out the contract's `_initialized` slot.

3. Upgrade the proxy to its actual implementation.

The contract's initialized slot is zeroed out as shown:

```
// Otherwise, we need to reset the initialized slot and call the initializer.  
// Reset the initialized slot by zeroing the single byte at `_offset` (from the right).  
bytes32 current = IStorageSetter(_target).getBytes32(_slot);  
uint256 mask = ~(uint256(0xff) << (uint256(_offset) * 8));  
IStorageSetter(_target).setBytes32(_slot, bytes32(uint256(current) & mask));
```

As seen from above, only one byte of the entire slot is zeroed out. As such, if `_initialized` (i.e. the initialized value) is ever greater than 255, this will not work since only one byte is zeroed out.

However, note that this isn't an issue in practice since all L1 contracts seem to inherit Openzeppelin v4.7.0 which has `_initialized` as `uint8`; only some L2 contracts inherit Openzeppelin v5 which uses `uint64`.

Recommendation: For future upgrades, ensure L1 contracts always inherit `Initializable` from Openzeppelin v4.7.0.

OP Labs: Fixed in PR 20289 and PR 20371.

Cantina Managed: Verified. The `Initializable` upgrade limitations have been explicitly documented.